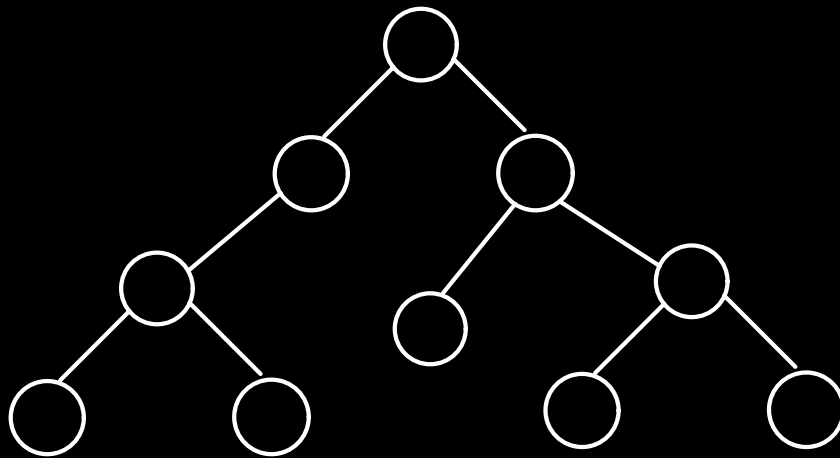
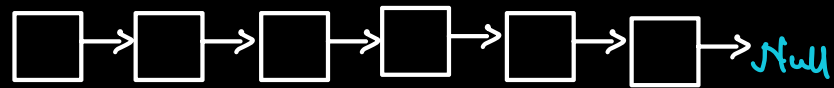


Graphs → Network



Social Media

DOM

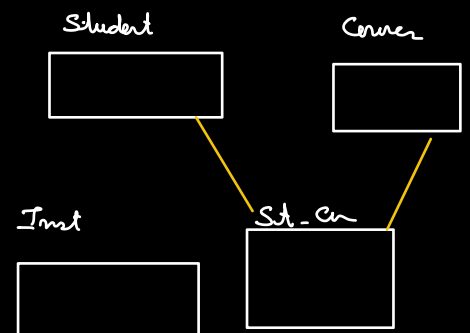
RDBMS

WWW

NN

Map

- Electric network
- Water pipeline
- Gas pipeline
- Roadways
- Railways
- Flight Network



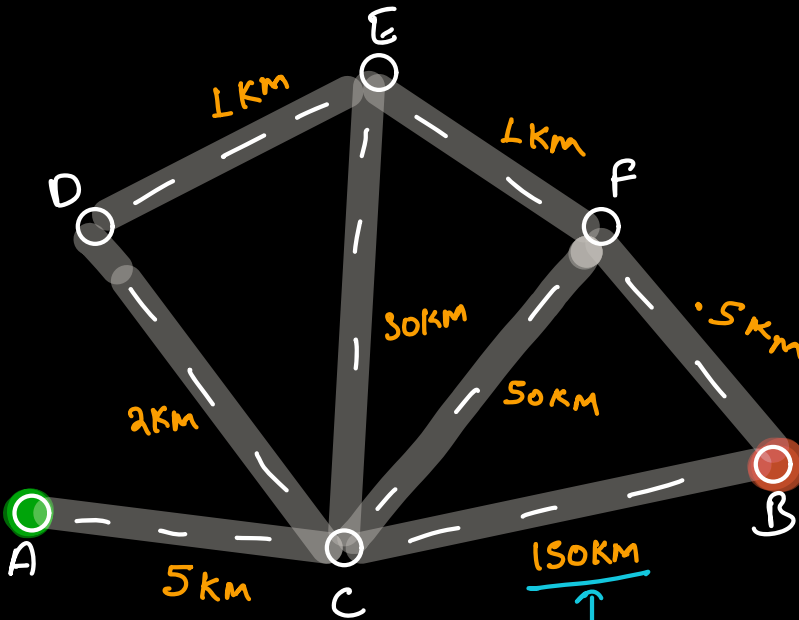
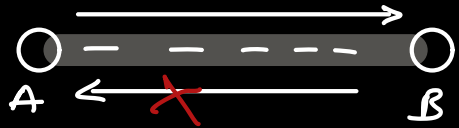
Unidirectional Graph

Linked In



Directed Graph

Instagram / X



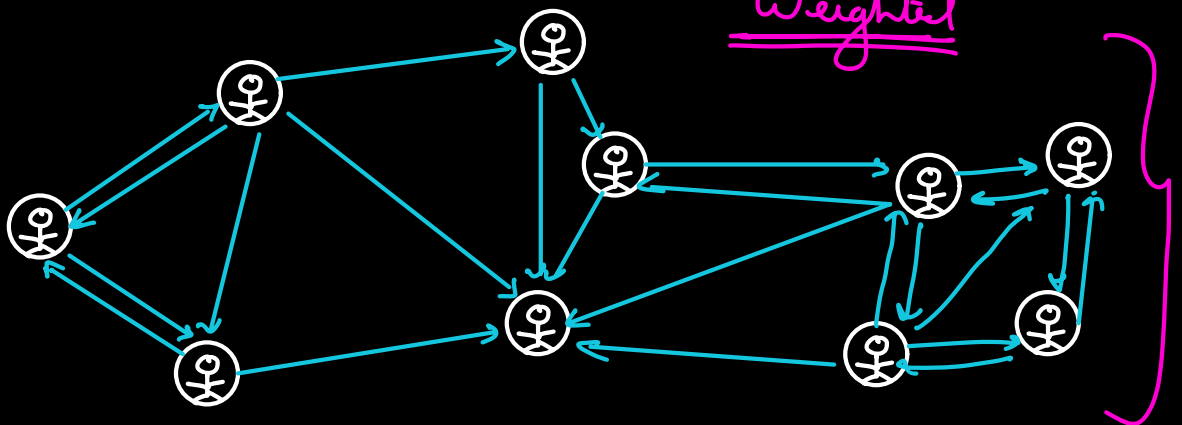
Weighted Graph

Weight

150+
200+
500+

10-20

Weighted

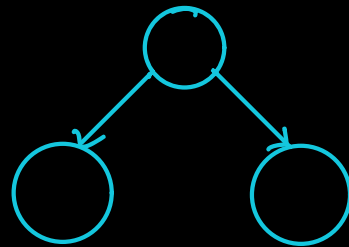


Binary Tree

```
class TreeNode {
```

```
    int data;  
    TreeNode left;  
    TreeNode right;
```

```
}
```

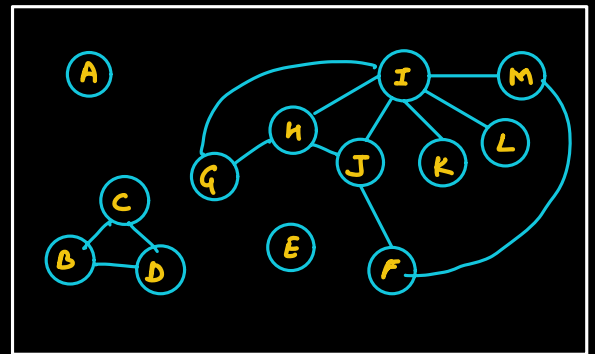


```
class GraphNode {
```

```
    int data;  
    Arraylist <GraphNode> nodes;
```

```
}
```

X



HashMap

Key

Node

A

B

C

D

E

F

G

⋮

Value

list of Neighbors

<>

<C, D>

<B, D>

<B, C>

<>

<J, M>

<I, H>

⋮

Adj list

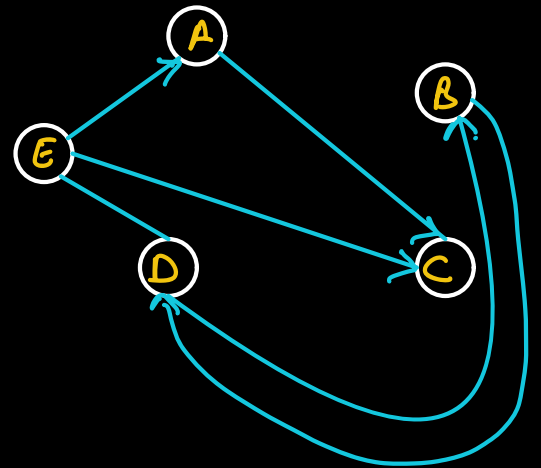
$V \rightarrow$ No. of Nodes
 $E \rightarrow$ No. of edges

```

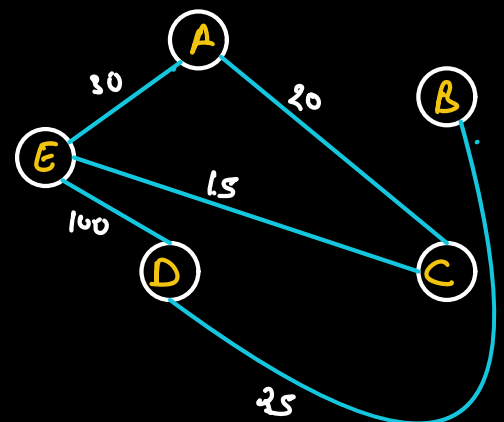
graph TD
    A((A)) --- C((C))
    A((A)) --- E((E))
    B((B)) --- C((C))
    C((C)) --- D((D))
    D((D)) --- E((E))

```

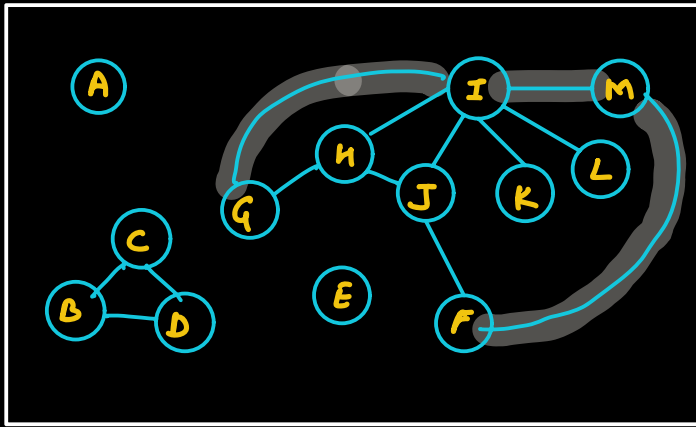
	A	B	C	D	E
A			T		F
B				T	
C	F				F
D		T			T
E	T		T	T	



	A	B	C	D	E
A			20		30
B				25	
C	20				15
D		25			100
E	30		15	100	



Q Given a graph & a source (S) and a dest (D).
Check if there exists a path from S to D.



S: F
D: G } True

S: B
D: G } False

<M, J>
 ↑
 G? X

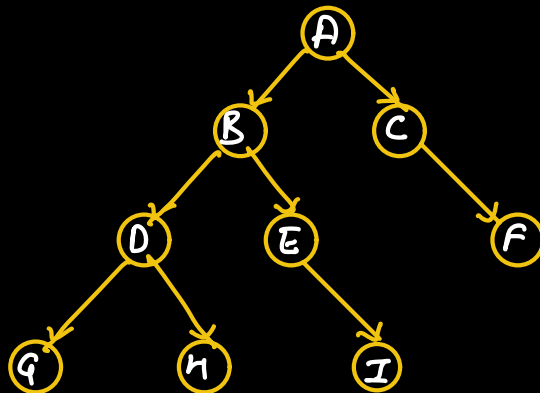
Binary Tree

S: B

D: H ✓

S: D

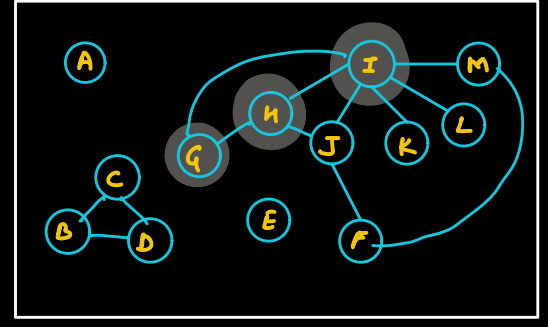
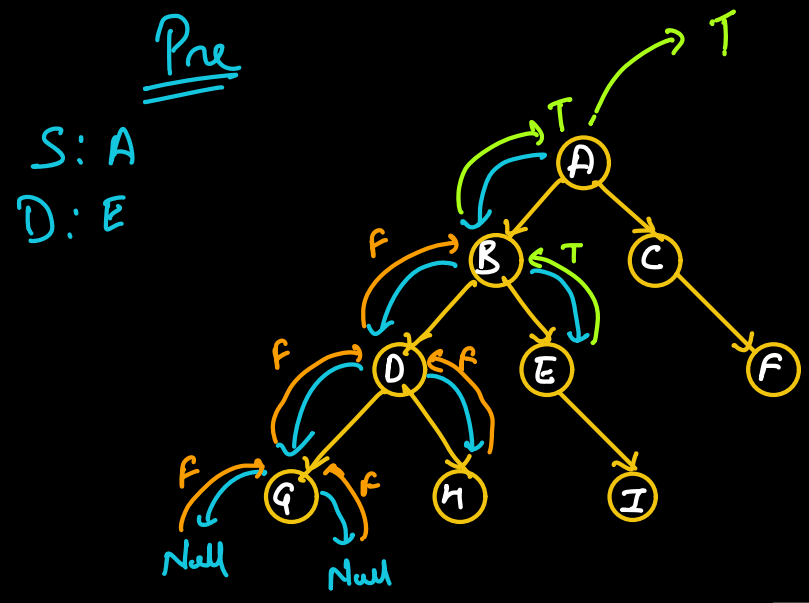
D: C X



```
boolean search (root, dest) {
    if (root == null) return false;
    if (root == dest) return true;
```

```
    return search (root.left, dest) ||
           search (root.right, dest);
}
```

level order <u>BFS</u>	<u>Pre</u> , <u>post</u> , <u>in order</u> <u>DFS</u>
---------------------------	--



```
boolean dfs(src, dest, visited[]) {
```

```
    visited[src] = true;  
    if (src == dest) return true;
```

```
    for (all nodes v in adj(src)) {
```

```
        if (!visited[v] && dfs(v, dest))
```

```
    }  
    return true;
```

```
}  
return false;
```

